PROJECT 02

Secure Multi-Department Campus Network with Security Automation

A practical technical guide, build record, and learning workbook

This guide explains the design, configuration, verification, Python configuration-auditing script, tests, results, and limitations of the project. It is written so the repository owner can review the work later and another learner can follow the same concepts in a controlled lab.

Repository: github.com/sayedmunibshah/secure-campus-network-security-automation

Platform: Cisco Packet Tracer + Python 3 + Git/GitHub

Document generated locally with Python and ReportLab

How to Use This Guide

Read the sections in order if you are learning campus-network design for the first time. If you are revising the project, use the command and troubleshooting sections as a checklist. Commands are examples for the Packet Tracer Cisco IOS-style devices used in this lab; exact command support can vary by device model and simulator version.

Learning path

- Understand the topology and the reason for separating departments into VLANs.
- Learn the IP plan and how each PC reaches its default gateway.
- Review switch access ports, trunk links, SVIs, and inter-VLAN routing.
- Understand management access, port security, and guest-isolation ACL intent.
- Reproduce verification tests and record the actual results.
- Run the Python auditor, inspect JSON reports, and run the seven unit tests.
- Review the limitations before describing the project publicly or extending it.

Accuracy note: this guide describes the latest project details available when the script was prepared. It does not read the current Packet Tracer file or GitHub repository automatically. If the project changes, update the guide and regenerate the PDF.

1. Project Overview

The goal was to build a small multi-department campus network in Cisco Packet Tracer and practise the controls commonly used on managed switches. The network separates Admin, IT, Staff, Guest, and Management traffic with VLANs. A multilayer switch provides the VLAN gateway interfaces and routing. Access switches connect end devices. A Python script checks selected security-related lines in saved Cisco configuration text and writes machine-readable JSON reports.

What the project demonstrates

- Logical network segmentation using VLANs and separate IP subnets.
- 802.1Q-style trunk links between access switches and the multilayer core.
- Inter-VLAN routing through switched virtual interfaces (SVIs).
- Basic switch hardening concepts: SSH management, port security, and unused-port shutdown.
- An access-control list (ACL) intended to limit Guest access to internal networks.
- A Python configuration auditor, JSON output, and automated unit tests.
- A Git-based project structure with evidence, configs, reports, tests, and documentation.

What it does not prove

A simulator lab is a learning environment, not a production security certification. Passing a ping test or receiving zero findings from a small script does not prove that the complete network is secure. The auditor checks only selected patterns in the supplied text; it does not connect to live switches, validate every IOS feature, or test the network end to end.

2. Network Topology and Device Roles

The design uses one multilayer core switch and two access switches. The core is the Layer 3 point for the VLAN gateways. The access switches connect PCs and carry the department VLANs upstream to the core.

Layer / device	Connections	Main role
SW-Core — Cisco 3560 multilayer switch	Gi0/1 to SW-Admin; Gi0/2 to SW-Staff	Core switching, VLAN gateways (SVIs), inter-VLAN routing, central ACL location for intended Guest policy
SW-Admin — Cisco 2960 access switch	Gi0/1 uplink; Fa0/1 PC-Admin; Fa0/2 PC-IT	Connects Admin and IT endpoints; management IP 192.168.99.2
SW-Staff — Cisco 2960 access switch	Gi0/1 uplink; Fa0/1 PC-Staff; Fa0/2 PC-Guest	Connects Staff and Guest endpoints; management IP 192.168.99.3
PC-Admin	SW-Admin Fa0/1	Admin endpoint in VLAN 10
PC-IT	SW-Admin Fa0/2	IT endpoint in VLAN 20
PC-Staff	SW-Staff Fa0/1	Staff endpoint in VLAN 30
PC-Guest	SW-Staff Fa0/2	Guest endpoint in VLAN 40; should not reach restricted internal subnets

Physical and logical links

```
SW-Core Gi0/1  &lt;---- trunk ----&gt;  SW-Admin Gi0/1
SW-Core Gi0/2  &lt;---- trunk ----&gt;  SW-Staff Gi0/1
SW-Admin Fa0/1 &lt;---- access ----&gt; PC-Admin  (VLAN 10)
SW-Admin Fa0/2 &lt;---- access ----&gt; PC-IT     (VLAN 20)
SW-Staff Fa0/1 &lt;---- access ----&gt; PC-Staff  (VLAN 30)
SW-Staff Fa0/2 &lt;---- access ----&gt; PC-Guest  (VLAN 40)
```

An access link normally carries traffic for one assigned access VLAN. A trunk link can carry traffic for multiple VLANs between switches. The VLAN ID in the Ethernet frame helps the switches keep the logical networks separate across shared uplinks.

3. VLANs and IP Addressing

A VLAN creates a separate Layer 2 broadcast domain. Each routed VLAN in this design also has its own IPv4 subnet. Hosts in the same VLAN can communicate at Layer 2 when their switching and addressing are correct. Hosts in different VLANs need a Layer 3 routing function, and policy may further restrict that communication.

VLAN	Name / purpose	Subnet	Default gateway
10	ADMIN	192.168.10.0/24	192.168.10.1
20	IT	192.168.20.0/24	192.168.20.1
30	STAFF	192.168.30.0/24	192.168.30.1
40	GUEST	192.168.40.0/24	192.168.40.1
99	MANAGEMENT	192.168.99.0/24	192.168.99.1
999	PARKING / unused ports	No host subnet configured	None

Endpoint addresses

Endpoint	IPv4 address	Mask	Gateway	VLAN
PC-Admin	192.168.10.10	255.255.255.0	192.168.10.1	10
PC-IT	192.168.20.10	255.255.255.0	192.168.20.1	20
PC-Staff	192.168.30.10	255.255.255.0	192.168.30.1	30
PC-Guest	192.168.40.10	255.255.255.0	192.168.40.1	40

The /24 mask (255.255.255.0) means the first three octets identify the subnet in this plan. For example, PC-Admin at 192.168.10.10 uses 192.168.10.1 as its gateway to reach destinations outside 192.168.10.0/24. The management addresses are 192.168.99.2 on SW-Admin and 192.168.99.3 on SW-Staff; the core management SVI is intended to

use 192.168.99.1.

Why VLAN 999 exists

VLAN 999 is used as a parking VLAN for unused ports. Unused access ports are assigned to it and administratively shut down in the documented core configuration. This reduces the chance that an unused port is accidentally used as a normal production access port. A parking VLAN is not a substitute for disabling unused ports or applying a complete access policy.

4. How the Network Works

4.1 Access ports

An access port connects an end device and is assigned to one VLAN. In this lab, PC-Admin and PC-IT connect to access ports on SW-Admin; PC-Staff and PC-Guest connect to access ports on SW-Staff. If a port is assigned to the wrong VLAN, the endpoint may have an incorrect subnet or be placed in the wrong security zone.

```
! Example concept — verify syntax against the device
interface FastEthernet0/1
 switchport mode access
 switchport access vlan 10
```

4.2 Trunk links

A trunk carries frames for multiple VLANs between switches. The project trunks are SW-Core Gi0/1 to SW-Admin Gi0/1 and SW-Core Gi0/2 to SW-Staff Gi0/1. The intended allowed VLAN list is 10, 20, 30, 40, 99, and 999. Restricting the allowed VLAN list to only what is needed is a useful hardening practice.

```
! Example verification
show interfaces trunk
show interfaces gigabitEthernet0/1 switchport
show interfaces gigabitEthernet0/2 switchport
```

4.3 SVIs and inter-VLAN routing

An SVI is a logical Layer 3 interface associated with a VLAN, commonly configured as interface vlan 10, interface vlan 20, and so on. A host uses its VLAN's SVI address as its default gateway. The core has ip routing enabled, so the multilayer switch can route between its directly connected VLAN subnets. Routing capability alone does not mean all cross-VLAN traffic should be allowed; ACLs or other policy should define permitted communication.

```
! Core verification commands
show ip interface brief
show ip route
show running-config | include ip routing
show running-config | section interface Vlan
```

For an SVI to become operational, the VLAN must exist and there generally must be at least one active Layer 2 port in that VLAN in an up state. Simulator behaviour can vary, so check both VLAN status and interface status instead of assuming that an SVI is up just because it appears in the configuration.

4.4 SSH management

SSH encrypts the remote management session, unlike Telnet which sends session data without equivalent transport encryption. The access switches are configured for SSH version 2, local VTY authentication, and SSH remote access. A production design should also use unique credentials, appropriate privilege boundaries, management-plane restrictions, and secure key handling. Never publish passwords, password hashes, private keys, or other secrets in the repository.

```
! Useful checks (exact support depends on IOS image)
show ip ssh
show running-config | section line vty
show running-config | include username|ip domain-name|crypto key
```

4.5 Port security

Port security limits which source MAC addresses can be learned on a switch port. The project access ports use a maximum of one secure MAC address, sticky learning, and the restrict violation action. Sticky learning allows a learned MAC address to be added to the running configuration. The restrict action drops traffic from a violating MAC and may increment counters or generate a log, depending on platform support. It is not a replacement for physical security, endpoint controls, or monitoring.

```
! Useful checks
show port-security
show port-security interface FastEthernet0/1
show port-security address
show interfaces status
```

4.6 Guest ACL intent

The ACL named GUEST-IN is intended to deny Guest-subnet traffic to Admin (192.168.10.0/24), IT (192.168.20.0/24), Staff (192.168.30.0/24), and Management (192.168.99.0/24), then permit other Guest traffic. The policy intent is sensible for a basic guest-isolation exercise, but an ACL must be attached at the correct interface and direction to have the intended effect. ACL names and definitions alone do not enforce a policy.

> Important current finding: the supplied SW-Core configuration has the GUEST-IN ACL defined but it was not shown as attached inbound to interface VLAN 40. The auditor therefore reports one informational finding for SW-Core. Do not describe the core ACL as confirmed effective until its attachment and end-to-end behaviour are verified.

The project also records that Guest pings to its own gateway succeeded while pings to internal gateways returned Destination host unreachable from 192.168.40.1. This is evidence of the observed test behaviour, not sufficient proof by itself that the intended ACL is correctly attached or that all guest traffic is isolated. The Staff switch also has an inbound Guest ACL on the Guest access port in the supplied configuration; review the actual simulator path and ACL counters before drawing a final conclusion.

5. Build Workflow

Step 1 — Build the topology

- Place one Cisco 3560 multilayer switch, two Cisco 2960 access switches, and four PCs.
- Connect the devices using the port map in Section 2.
- Use clear device labels: SW-Core, SW-Admin, SW-Staff, PC-Admin, PC-IT, PC-Staff, PC-Guest.
- Save the Packet Tracer project locally. The .pkt files are intentionally excluded from the public Git repository in this project version.

Step 2 — Configure VLANs and trunks

Create VLANs 10, 20, 30, 40, 99, and 999 on the required switches. Configure endpoint-facing ports as access ports in the appropriate VLAN. Configure the uplinks as trunks and confirm that the intended VLANs are allowed. Use show vlan brief and show interfaces trunk to verify rather than relying only on the configuration text.

Step 3 — Configure the Layer 3 core

On the multilayer core, configure the SVI gateway addresses and enable Layer 3 routing with ip routing. Verify that each expected SVI has the correct address and state. If a gateway is down, check VLAN existence, active access/trunk membership, and link state.

Step 4 — Configure endpoint addressing

Assign each PC its planned IP address, /24 mask, and VLAN gateway. First test each PC's own gateway. A failed gateway ping commonly points to local addressing, access VLAN, cabling, interface state, or SVI state before it points to a remote ACL.

Step 5 — Apply management and access controls

Configure SSH v2 and local VTY authentication on the switches, port security on active access ports, and shutdown unused ports. Apply the Guest ACL where the policy design requires it. Keep passwords and hashes out of screenshots, config exports, and Git history.

Step 6 — Test, document, and audit

Run connectivity and policy tests, capture useful evidence, export sanitized configuration text, run the Python auditor, run the unit tests, inspect JSON reports, and update README.md with the actual results and known limitations. A test result should be recorded as observed rather than rewritten to match the expected outcome.

6. Cisco Verification and Troubleshooting

The commands below are a practical verification checklist. Run privileged EXEC commands on the relevant switch. Packet Tracer device models may support only a subset of real IOS commands.

Question	Command	What to inspect
Which VLANs exist?	show vlan brief	VLAN IDs, names, and access-port membership
Are uplinks trunking?	show interfaces trunk	Trunk status, native VLAN, allowed and active VLANs
Are interfaces up?	show ip interface brief	SVI addresses and up/down state
Are routes present?	show ip route	Connected routes for the VLAN subnets
Is routing enabled?	show running-config | include ip routing	Presence of the routing command on the core
Is SSH enabled?	show ip ssh	SSH version and operational status, if supported
What is on VTY lines?	show running-config | section line vty	Authentication and permitted remote transport
Is port security configured?	show port-security	Enabled ports, maximum addresses, violations
What ACLs are defined?	show access-lists	ACL entries and match counters, if supported
Is ACL applied to an interface?	show ip interface vlan 40	Inbound/outbound access list attachment, if supported
What config is active?	show running-config	Current configuration; sanitize before sharing
Was config saved?	show startup-config	Saved startup configuration, if supported

A sensible troubleshooting order

- Check the PC IP address, mask, and default gateway.
- Check physical links and switch port state.
- Check the endpoint port's access VLAN.
- Check that the VLAN exists and the uplink trunk carries it.
- Check the core SVI state and IP address.
- Check the routing table and the intended route between source and destination.
- Check ACL definition, interface attachment, direction, rule order, and counters.
- Repeat the test and capture the exact result, source, destination, and time.

A successful ping to a host's own gateway verifies only a portion of the path. It does not prove that other VLANs are reachable or that a security policy is effective. Likewise, a failed ping can be caused by an ACL, incorrect addressing, a down interface, or ICMP being filtered; use multiple checks to isolate the cause.

7. Python Configuration Auditor

Purpose and scope

The script at scripts/audit_config.py reads a Cisco IOS-style configuration text file, performs a limited set of pattern-based checks, prints findings, and can save a JSON report. It is a static text auditor: it does not log in to switches, execute show commands, validate the live configuration, or prove the full security posture of a network.

Selected checks

- SSH version 2 is explicitly configured.
- HTTP management service is enabled.
- VTY remote-access transport and authentication settings are checked.
- Port security is checked on selected active access ports.
- Port-security violation action is checked.
- Selected interface conditions are checked.
- An inbound ACL is expected on the Guest access port.
- ACLs that are defined but not shown attached are reported.

How interface sections are parsed

The script uses a regular expression to split the configuration into interface blocks. The current parser expression is:

```
r'(?ms)^interface\s+(\S+)\s*\n(.*?)(?=^[^\s]|\Z)'
```

In broad terms, it looks for a line beginning with interface, captures the interface name, then captures the following indented configuration until the next top-level configuration line or the end of the text. This is a practical parser for the expected IOS-style input, not a complete Cisco configuration grammar. Different indentation, nested sections, or platform-specific syntax may require improvements.

Run the auditor

From the project root, run the help command first:

```
python3 scripts/audit_config.py --help
```

Generate one JSON report for each sanitized device configuration:

```
python3 scripts/audit_config.py configs/SW-Admin.cfg --device SW-Admin --output reports/SW-Admin-audit.
json
python3 scripts/audit_config.py configs/SW-Core.cfg --device SW-Core --output reports/SW-Core-audit.json
python3 scripts/audit_config.py configs/SW-Staff.cfg --device SW-Staff --output reports/SW-Staff-audit.
json
```

The input is a text configuration file. The --device option labels the report, and --output specifies where the JSON result is written. Inspect the JSON files after each run and confirm that they match the configuration revision you intended to audit.

Example JSON shape

The exact field names and findings depend on the current script, but a JSON audit report is generally useful because tools can read it consistently and a later workflow can aggregate results. A simplified illustration—not a copy of a guaranteed exact report schema—is shown below:

```
{
  "device": "SW-Core",
  "findings": [
    {
      "id": "ACL-001",
      "severity": "INFO",
      "message": "ACL guest-in defined but not shown attached"
    }
  ]
}
```

Use the actual reports in reports/ as the source of truth for exact JSON fields. The example above is illustrative. Do not edit a report manually to make it appear cleaner; rerun the auditor after fixing or changing a configuration extract.

How to interpret results

A finding is a prompt for investigation. An informational finding may indicate a configuration that should be reviewed, not necessarily an immediately exploitable vulnerability. Zero findings means only that the checks currently implemented did not detect their target patterns in that input. It does not mean the device or network is fully secure.

8. Unit Tests

The test file is tests/test_audit_config.py. Seven tests were reported as passing in the latest recorded run. The tests exercise selected auditor behaviours and help prevent regressions when the script is changed.

Test	What it checks
test_accepts_guest_port_with_inbound_acl	A Guest access port with an inbound ACL should be accepted by the relevant check.
test_clean_config_has_no_findings	A sample configuration meeting the checks should produce no findings.
test_detects_access_port_without_port_security	The auditor should flag an access port missing port-security configuration.
test_detects_disabled_vty_transport	The auditor should detect an unsuitable or disabled VTY transport configuration.
test_detects_guest_port_without_inbound_acl	The auditor should flag the Guest port when the expected inbound ACL is absent.
test_detects_http_server	The auditor should identify HTTP management service enabled in the input.
test_reports_device_name	The report should include the expected device label.

Run the tests

```
python3 -m unittest discover -s tests -v
```

The recorded output showed all seven tests passing, with the final status OK. A passing unit-test suite verifies the tested behaviours in the code; it does not validate the physical or simulated network itself. When changing the auditor, add tests for new checks and for false-positive/false-negative cases.

9. Latest Recorded Audit Results

Device	Recorded result	Interpretation
SW-Admin	0 findings	No issue was detected by the currently implemented checks in the supplied sanitized configuration extract.
SW-Core	1 informational finding	ACL-001: GUEST-IN is defined but was not shown attached in the supplied core configuration.
SW-Staff	0 findings	No issue was detected by the currently implemented checks in the supplied sanitized configuration extract.

The latest recorded test run reported seven tests passing. These results are a snapshot from the project notes available when this guide was authored. Re-run the auditor and unit tests if the configuration or script changes; update this section to match the new output.

Evidence to retain

- A readable topology screenshot with device labels and links.
- Screenshots or text output for VLANs, trunks, SVIs, and routing.
- Representative port-security and SSH verification output.
- Connectivity results showing both successful and blocked tests.
- The three JSON audit reports and the complete unit-test output.

Before publishing evidence, inspect every screenshot and configuration for usernames, passwords, hashes, keys, public IPs, or other sensitive values. Prefer sanitized configurations and avoid publishing secrets even if they are hashed.

10. GitHub Repository and Files

The public repository is intended to keep the project understandable and reviewable. The Packet Tracer .pkt files are excluded from Git in the current version, so the repository documents the design and supporting artifacts but may not contain the full editable simulation file.

```
secure-campus-network-security-automation/
■■■ .gitignore
■■■ README.md
■■■ configs/
■   ■■■ SW-Admin.cfg
■   ■■■ SW-Core.cfg
■   ■■■ SW-Staff.cfg
■■■ evidence/
■   ■■■ Secure-Multi-Department-Campus-Network.png
■   ■■■ SW-Admin.png
■   ■■■ SW-Core.png
■   ■■■ SW-Staff.png
■   ■■■ admin-connectivity-tests.png
■   ■■■ guest-isolation-tests.png
■■■ reports/
■   ■■■ SW-Admin-audit.json
■   ■■■ SW-Core-audit.json
■   ■■■ SW-Staff-audit.json
■■■ scripts/
■   ■■■ audit_config.py
■■■ tests/
    ■■■ test_audit_config.py
```

The evidence directory may contain additional numbered screenshots beyond the simplified tree shown above. The file tree is an explanatory overview, not a guarantee that every filename is listed. The local helper create_readme.py was left untracked in the recorded status and is not part of the committed project tree.

Useful Git commands

```
git status
git log --oneline -5
git add README.md
git commit -m "Improve README with project details and limitations"
git push
```

Only run the add/commit/push commands when you have reviewed the changes and are ready to publish them. If Git reports that there is nothing to commit, check git status rather than creating an unnecessary commit. Never commit the local Packet Tracer file or secrets unless you have deliberately reviewed the repository policy and the file is safe to share.

11. Lessons Learned

- VLANs separate Layer 2 broadcast domains, but routing and ACL policy determine what can communicate between them.
- A trunk must carry the required VLANs; a misconfigured trunk can make an otherwise correct VLAN design fail.
- A configured SVI is not necessarily operational. VLAN existence and active Layer 2 membership matter.
- An ACL definition does not enforce policy unless it is attached to the correct interface and direction.
- A successful ping to the default gateway does not prove the complete path or isolation policy.
- Static configuration auditing can catch repeatable issues, but a pattern-based script has a limited view of the device.
- Unit tests check code behaviour; they are not a replacement for network verification.
- Honest limitations and reproducible evidence make a portfolio project more credible than unsupported claims.
- Git makes the work reviewable when the repository separates source code, configs, test evidence, and reports.

12. Limitations and Future Improvements

Known limitations

- The GUEST-IN ACL was defined in the supplied core configuration but was not shown attached to the VLAN 40 SVI; this remains a verification item.
- Guest ping failures alone do not establish that all guest traffic is isolated or that the policy is attached correctly.
- The auditor checks selected patterns in supplied text and does not inspect live devices or the complete running state.
- A zero-finding report does not establish that a configuration is fully secure.
- The parser supports the expected IOS-style configuration format, not every Cisco syntax or platform variant.
- The .pkt simulation files are local and excluded from the public repository in the current setup.
- The lab does not include production identity management, centralized logging, monitoring, redundancy, or a comprehensive threat model.

Recommended next steps

- Resolve and verify the Guest ACL attachment on the intended Layer 3 interface; check interface output and ACL counters, then test both allowed and denied paths.
- Create a repeatable test matrix: source VLAN, destination, protocol, expected result, actual result, and evidence filename.
- Expand auditor coverage for unused-port state, trunk allowed VLANs, native VLAN configuration, management-plane restrictions, and weak or missing authentication settings where safely detectable.

- Add tests for malformed configurations, missing interface sections, duplicate ACL names, rule ordering, and cases where the parser should avoid false positives.
- Add a report summary that counts findings by severity and records the input filename or a hash so results can be tied to a particular config revision.
- Consider safe lab-only automation that collects read-only verification outputs, with credentials kept outside source control.
- Add a diagram, a test matrix, and a clear 'verified / not yet verified' status to the README.
- If sharing the Packet Tracer file becomes practical, review simulator version compatibility and remove all credentials or sensitive configuration first.

13. Responsible Use and Security Hygiene

This is a defensive learning lab. Test configuration changes only in your own simulation or an environment where you have explicit permission. Do not use these commands to access devices or networks without authorization.

- Use unique lab credentials and rotate any credential that may have been exposed.
- Never publish passwords, hashes, private keys, tokens, or confidential network details.
- Sanitize configuration exports before placing them in GitHub.
- Keep a backup of a known-good lab configuration before experimenting.
- Record what you actually observed and clearly label unverified assumptions.

14. Quick Revision Sheet

Concept	Remember
VLAN	Separates Layer 2 broadcast domains.
Access port	Normally carries one assigned endpoint VLAN.
Trunk	Carries multiple VLANs between network devices.
SVI	Logical Layer 3 interface for a VLAN; often the default gateway.
Inter-VLAN routing	Layer 3 forwarding between different VLAN subnets.
ACL	Ordered rules that permit or deny matching traffic; attachment and direction matter.
SSH	Encrypted remote CLI management; preferable to Telnet.
Port security	Limits learned/allowed MAC addresses on a switch port.
JSON report	Structured output that can be inspected or consumed by other tools.
Unit test	Checks a specific software behaviour, not the entire network.

Five questions to ask after every change

- What behaviour did I intend to change?
- Which configuration line or topology element controls that behaviour?
- Which command or test verifies the change?
- What did the test actually show?
- What evidence and limitations should I record?

Appendix A — Commands Used in the Python Workflow

Run these commands from the project root, ~/Projects/02-Secure-Campus-Network. If your local directory differs, change into the directory that contains README.md, scripts/, reports/, and tests/.

```
cd ~/Projects/02-Secure-Campus-Network

# Review the auditor's options
python3 scripts/audit_config.py --help

# Run the three device audits
python3 scripts/audit_config.py configs/SW-Admin.cfg --device SW-Admin --output reports/SW-Admin-audit.
json
python3 scripts/audit_config.py configs/SW-Core.cfg --device SW-Core --output reports/SW-Core-audit.json
python3 scripts/audit_config.py configs/SW-Staff.cfg --device SW-Staff --output reports/SW-Staff-audit.
json

# Run all unit tests verbosely
python3 -m unittest discover -s tests -v

# Inspect changed files before publishing
git status
git diff -- README.md
```

Appendix B — Glossary

Term	Meaning
ACL	Access Control List: ordered permit/deny rules for matching traffic.
Default gateway	Router or Layer 3 interface a host uses for off-subnet traffic.
IOS	Cisco Internetwork Operating System; syntax varies by platform and release.
JSON	JavaScript Object Notation; a structured text data format.
MAC address	Layer 2 hardware address used in Ethernet forwarding.
SVI	Switched Virtual Interface: a logical interface associated with a VLAN.
VTY	Virtual terminal lines used for remote CLI sessions.
/24	CIDR prefix equivalent to subnet mask 255.255.255.0.

Final Note

This project is a useful foundation for further work in switching, segmentation, network access control, configuration automation, and test-driven tooling. The strongest next step is not simply adding more features: it is closing the known Guest ACL verification gap, recording reproducible evidence, and extending the auditor with tests for each new rule.

Project repository: secure-campus-network-security-automation